



Đại học Khoa học Tự nhiên

Giải thuật di truyền trong bài toán người du lịch

Sinh viên :

Nguyễn Thùy Trang - 22000128

Phạm Thị Duyên - 22000079

Giảng viên :

LÊ HUY HOÀNG

Ngày 30 tháng 12 năm 2024

Mục lục

1	Lời nói đầu	5
1.1	Tính cấp thiết của đề tài	5
1.2	Mục đích nghiên cứu	5
1.3	Đối tượng nghiên cứu	5
1.4	Phạm vi nghiên cứu	5
1.5	Nội dung nghiên cứu	5
2	Bài toán người du lịch (TSP - Travelling Salesman Problem)	6
2.1	Lịch sử hình thành bài toán	6
2.2	Phát biểu bài toán	7
2.3	Phân tích độ phức tạp bài toán	7
2.4	Ứng dụng thực tiễn của bài toán người du lịch	8
3	Giải thuật di truyền (GA - Genetic Algorithms)	9
3.1	Nguyên lý hoạt động	9
3.2	Các toán tử di truyền	11
3.2.1	Toán tử lai ghép	11
3.2.2	Toán tử đột biến	11
3.2.3	Toán tử chọn lọc	12
3.2.4	Một số toán tử khác	13
3.3	Một số biến thể của thuật toán di truyền	13
3.3.1	Thuật toán di truyền mã hóa thực và nhị phân	13
3.3.2	Thuật toán di truyền đa mục tiêu	14
3.3.3	Thuật toán di truyền song song	15
3.3.4	Thuật toán di truyền hỗn loạn	16
3.3.5	Thuật toán di truyền lai ghép	16
4	Thực nghiệm	17
4.1	Bộ dữ liệu	17
4.2	Các bước xây dựng giải thuật GA	17

5	Đánh giá và kết luận	20
5.1	So sánh hiệu quả giữa các phép lai ghép	21
5.2	Một số phương án kết hợp đề xuất	21
5.2.1	Thuật toán tìm kiếm Tabu	21
5.2.2	Tìm kiếm địa phương kết hợp	22

Lời cảm ơn

Đầu tiên chúng em muốn bày tỏ lòng biết ơn sâu sắc đến *Trường Đại học Khoa học Tự nhiên, Đại học quốc gia Hà Nội* và đặc biệt là các thầy cô giáo trong khoa *Toán Cơ Tin học*. Trong suốt thời gian học vừa qua, sự hướng dẫn và kiến thức quý báu mà các thầy cô đã truyền đạt đã làm cho chúng em trưởng thành hơn về chuyên môn, về nhân cách và về đạo đức. Chúng em xin chân thành cảm ơn thầy giáo, *TS Lê Huy Hoàng*, người đã tận tình hướng dẫn, giúp đỡ chúng em trong suốt quá trình thực hiện và hoàn thành bài nghiên cứu này. Thầy không chỉ cung cấp tài liệu mà còn truyền đạt những phương pháp và định hướng nghiên cứu quan trọng. Chúng em cũng muốn gửi lời cảm ơn chân thành đến gia đình và bạn bè, những người đã luôn ủng hộ, động viên và giúp đỡ em trong quá trình học tập và nghiên cứu. Cuối cùng, chúng em trân trọng biết ơn và đánh giá cao những kinh nghiệm quý báu mà chúng em đã được học từ các thầy cô và những người xung quanh. Những kiến thức và phẩm chất mà em đã tích lũy trong thời gian học tập này sẽ luôn là tài sản quan trọng và là nguồn động lực để em tiếp tục phát triển trong cuộc sống và sự nghiệp.

Bảng ký hiệu

TSP	Bài toán người du lịch.
GA	Giải thuật di truyền.
GAs	Các giải thuật di truyền (viết tắt của Genetic Algorithms).
NP	Bài toán giải trong thời gian đa thức hoặc nghiệm kiểm tra trong đa thức.
NP-hard	Bài toán khó hơn hoặc tương đương mọi bài toán trong NP.
Schemas	Cấu trúc trong GA biểu diễn thông tin giải pháp.
Chromosome	Nhiễm sắc thể, đại diện lời giải trong GA.
Gene	Gen, phần tử của nhiễm sắc thể, đại diện thành phần lời giải.
Allele	Alen, giá trị cụ thể của gen.
Population	Quần thể, tập hợp các nhiễm sắc thể trong thế hệ.
Fitness	Hàm đánh giá chất lượng lời giải.

1 Lời nói đầu

1.1 Tính cấp thiết của đề tài

Bài toán người du lịch (Traveling Salesman Problem - TSP) là một trong những bài toán tối ưu tổ hợp cổ điển nhất trong lĩnh vực khoa học máy tính và toán học ứng dụng. Với nhiều ứng dụng thực tiễn như quy hoạch lộ trình giao hàng, thiết kế mạng giao thông, hay tối ưu hóa trong sản xuất, việc giải quyết bài toán TSP đã trở thành một thách thức quan trọng. Trong báo cáo này, việc ứng dụng thuật toán di truyền được đề xuất như một phương pháp hiệu quả để giải quyết bài toán này.

1.2 Mục đích nghiên cứu

Mục tiêu chính của nghiên cứu là đánh giá khả năng ứng dụng thuật toán di truyền trong việc giải quyết bài toán TSP. Thông qua đó, nghiên cứu nhằm phát triển một phương pháp hiệu quả và tính ứng dụng cao, đóng góp vào kho tàng tri thức trong lĩnh vực tối ưu hóa và khoa học dữ liệu.

1.3 Đối tượng nghiên cứu

Đối tượng nghiên cứu trong đề tài này là bài toán người du lịch (TSP) cùng với các giải thuật sử dụng thuật toán di truyền. Ngoài ra, nghiên cứu còn đề cập đến các thông số và tham số ối định của thuật toán di truyền trong bài toán này.

1.4 Phạm vi nghiên cứu

Phạm vi nghiên cứu bao gồm việc nghiên cứu đề xuất, thử nghiệm và đánh giá hiệu quả của thuật toán di truyền trong việc giải quyết bài toán TSP. Nghiên cứu sẽ tập trung vào các bộ dữ liệu TSP có quy mô vừa và lớn, đồng thời đầu tư nghiên cứu vào việc cải tiến tham số để tăng hiệu suất tính toán và chất lượng giải pháp.

1.5 Nội dung nghiên cứu

Nội dung nghiên cứu của đề tài bao gồm các phần chính như sau:

- Tổng quan lý thuyết về bài toán TSP và các thuật toán giải.

- Nguyên lý và quy trình áp dụng thuật toán di truyền cho bài toán TSP.
- Thử nghiệm trên các bộ dữ liệu chuẩn, phân tích kết quả và so sánh hiệu quả.
- Đề xuất cải tiến thuật toán di truyền nhằm nâng cao chất lượng giải pháp và hiệu năng tính toán.

2 Bài toán người du lịch (TSP - Travelling Salesman Problem)

2.1 Lịch sử hình thành bài toán

Bài toán TSP được định nghĩa trong thế kỉ 19 bởi nhà toán học Ireland William Rowan Hamilton và nhà toán học Anh Thomas Kirkman. Trường hợp tổng quát của TSP có thể được nghiên cứu lần đầu tiên bởi các nhà toán học ở Vienna và Harvard trong những năm 1930 và là một trong những bài toán được nghiên cứu sâu nhất trong tối ưu hóa. Nó thường được dùng làm thước đo cho nhiều phương pháp tối ưu hóa. Mặc dù bài toán rất khó giải trong trường hợp tổng quát, có nhiều phương pháp giải chính xác cũng như heuristic đã được tìm ra để giải quyết một số trường hợp có tới hàng chục nghìn thành phố.

Nguồn gốc của bài toán này vẫn chưa được làm rõ, tuy nhiên, một cuốn sổ tay của người bán hàng xuất bản năm 1832 có đề cập tới bài toán này với ví dụ cho chu trình nước Đức và Thụy Sĩ, dù bên trong không có bất cứ công thức toán học nào.

Trong những năm 1950 và 1960, bài toán trở nên phổ biến trong giới nghiên cứu khoa học ở Châu Âu và Mỹ. Trong đó, George Dantzig, Delbert Ray Fulkerson và Selmer M. Johnson ở công ty RAND tại Santa Monica đã có đóng góp quan trọng cho bài toán này, bằng cách biểu diễn bài toán dưới dạng quy hoạch nguyên và đưa ra phương pháp mặt phẳng cắt để tìm ra lời giải. Với phương pháp mới này, họ đã giải được tối ưu một trường hợp có 49 thành phố bằng cách xây dựng một chu trình và chứng minh rằng không có chu trình nào ngắn hơn. Trong những thập niên tiếp theo, bài toán được nghiên cứu bởi nhiều nhà nghiên cứu trong các lĩnh vực toán học, khoa học máy tính, hóa học, vật lý, và các ngành khác.

Năm 1972, Richard M. Karp chứng minh rằng bài toán chu trình Hamilton là NP-đầy

đủ, kéo theo bài toán TSP cũng là NP-đầy đủ. Đây là một lý giải toán học cho sự khó khăn trong việc tìm kiếm chu trình ngắn nhất.

Một bước tiến lớn được thực hiện cuối thập niên 1970 và 1980 khi Grötschel, Padberg, Rinaldi và cộng sự đã giải được những trường hợp lên tới 2392 thành phố, sử dụng phương pháp mặt phẳng cắt và nhánh cận. Và cho đến nay, bài toán TSP vẫn được nghiên cứu để tìm lời giải cho các bộ dữ liệu lớn hơn.

2.2 Phát biểu bài toán

Bài toán người du lịch là một trong những bài toán tối ưu hóa tổ hợp nổi tiếng nhất trong lĩnh vực khoa học máy tính và nghiên cứu hoạt động. Bài toán yêu cầu tìm một hành trình ngắn nhất hoặc một chi phí nhỏ nhất để một người bán hàng đi qua tất cả các thành phố trong một tập các thành phố đã cho sao cho mỗi thành phố chỉ đi qua đúng một lần và trở về thành phố xuất phát ban đầu.

Phát biểu bài toán TSP như sau: Cho một đồ thị có trọng số đầy đủ $G = (V, E)$, trong đó V là tập hợp các thành phố và E là tập hợp các đường đi giữa các thành phố, mỗi đường đi có một trọng số không âm. Yêu cầu là tìm một chu trình Hamilton (một hành trình qua tất cả các thành phố một lần và trở về thành phố ban đầu) có tổng trọng số là nhỏ nhất.

2.3 Phân tích độ phức tạp bài toán

Độ phức tạp của bài toán TSP được xếp vào loại bài toán NP-đầy đủ (NP-complete), một trong những lớp bài toán khó nhất trong lý thuyết độ phức tạp tính toán.

TSP yêu cầu tìm một chu trình tối thiểu qua n thành phố sao cho mỗi thành phố được đi qua đúng một lần và trở về điểm xuất phát với tổng khoảng cách là nhỏ nhất. Với n thành phố, có tổng cộng $\frac{(n-1)!}{2}$ chu trình khác nhau cần phải xem xét nếu sử dụng phương pháp liệt kê tất cả các khả năng, dẫn đến độ phức tạp thời gian của bài toán theo hàm giai thừa.

Không có thuật toán nào được biết đến có thể giải bài toán TSP trong thời gian đa thức cho mọi trường hợp và bất kỳ giải pháp chính xác nào cho TSP cũng đòi hỏi thời gian tính toán tăng theo cấp số nhân với số lượng thành phố. Mặc dù không có lời giải chính xác nhanh chóng cho TSP, nhiều phương pháp gần đúng đã được phát triển để đưa

ra lời giải tốt trong thời gian hợp lý như: một số thuật toán Heuristic, thuật toán nhánh cận và mặt phẳng cắt, thuật toán Tabu,..

2.4 Ứng dụng thực tiễn của bài toán người du lịch

Phần lớn công trình nghiên cứu về bài toán người du lịch với mục đích làm nền tảng nghiên cứu các phương pháp chung có thể áp dụng cho nhiều vấn đề tối ưu hóa rời rạc. Thật vậy, nhiều ứng dụng trực tiếp của TSP mang lại giá trị cho lĩnh vực nghiên cứu như làm bài toán phụ để giải bài toán khó, trong học máy, phân tích Gen trong nghiên cứu sinh học,..

Ngoài ra, bài toán TSP có nhiều ứng dụng trong thực tế:

- Vận chuyển và hậu cần: trong lĩnh vực vận chuyển hàng hóa, bài toán TSP thường được sử dụng để tối ưu hóa hành trình của các xe vận chuyển. Bằng cách xác định một hành trình ngắn nhất để ghé thăm các điểm đến, doanh nghiệp có thể giảm thiểu chi phí vận chuyển và tăng hiệu quả hoạt động.
- Lập kế hoạch và lập lịch: bài toán TSP cũng được áp dụng trong lập kế hoạch và lập lịch công việc. Ví dụ, trong việc lập lịch công việc cho các kỹ sư hoặc kỹ thuật viên, bài toán TSP có thể giúp xác định một hành trình tối ưu để họ thăm các địa điểm khác nhau trong một khu vực nhất định.
- Định tuyến mạng: trong các mạng máy tính và mạng viễn thông, bài toán TSP được áp dụng để tối ưu hóa định tuyến dữ liệu. Bằng cách xác định hành trình ngắn nhất để truyền dữ liệu qua các nút mạng, hệ thống mạng có thể hoạt động hiệu quả hơn và giảm thiểu độ trễ.
- Tính toán tối ưu: bài toán TSP cũng có ứng dụng trong nhiều lĩnh vực tính toán tối ưu khác, bao gồm thiết kế mạch tích hợp, quy hoạch sản xuất, lập lịch sản xuất và tối ưu hóa các quá trình công nghiệp. Bằng cách tối ưu hóa hành trình hoặc sắp xếp các công việc, bài toán TSP có thể cải thiện hiệu quả và giảm thiểu chi phí trong quá trình này.

3 Giải thuật di truyền (GA - Genetic Algorithms)

Hiện tại vẫn chưa có thuật toán nào giải quyết chính xác tổng quát nhất các dạng bài toán NP-Hard trong thời gian đa thức. Vì vậy, các nhà nghiên cứu tìm đến các phương pháp xấp xỉ (heuristic và meta-heuristic) để tìm ra các lời giải gần tối ưu trong thời gian chấp nhận được. Các phương pháp này không đảm bảo tìm ra lời giải tốt nhất, nhưng thường tìm thấy các lời giải đủ tốt một cách nhanh chóng và dễ dàng. Thuật toán di truyền là một thuật toán tối ưu hoá trong lớp thuật toán MetaHeuristic, bao gồm những thuật toán được lấy ý tưởng từ tự nhiên như quá trình tiến hoá sinh học, tập tính bầy đàn hay định luật trong vật lý. J.H. Holland đã đề xuất GA vào năm 1992, lấy cảm hứng từ quá trình chọn lọc tự nhiên.

3.1 Nguyên lý hoạt động

Algorithm 1 Evolutionary Algorithm [1]

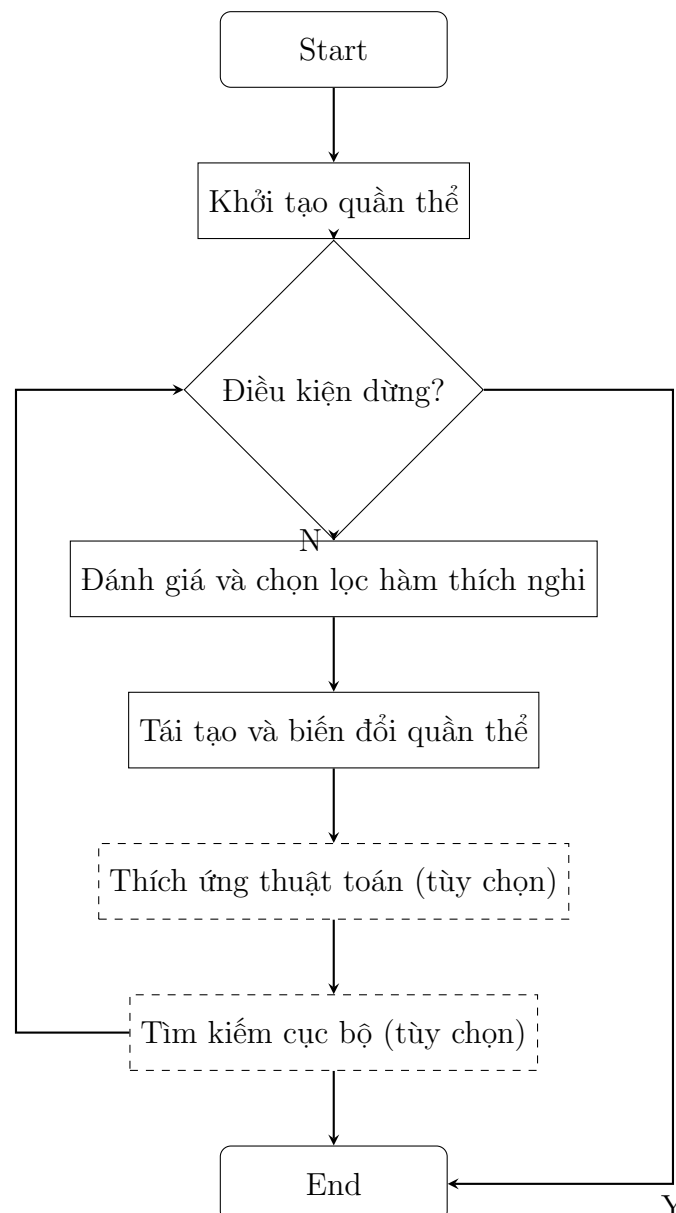
```
1:  $P(t)$ : quần thể các giải pháp;  $t$ : số lần lặp;  
2: begin  
3:    $t \leftarrow 0$   
4:   khởi tạo  $P(t)$   
5:   đánh giá  $P(t)$   
6:   while (điều kiện dừng chưa đạt) do  
7:     begin  
8:        $t \leftarrow t + 1$   
9:       chọn  $P(t)$  từ  $P(t - 1)$   
10:      biến đổi  $P(t)$   
11:      đánh giá  $P(t)$   
12:     end  
13: end = 0
```

Thuật toán chung của một giải thuật di truyền theo các bước [2]:

- + Chọn một quần thể ban đầu $x_0 = \{x_{10}, x_{20}, \dots, x_{N0}\}$, $x_i \in S$, trong đó S là không gian tìm kiếm;
- + Xác định giá trị hàm mục tiêu $f(x_0)$ cho mỗi cá thể của quần thể;
- + Lặp lại cho mọi lần lặp j cho đến khi đạt điều kiện dừng;
- + Thực hiện chọn lọc;

- + Thực hiện lai ghép với một xác suất;
- + Thực hiện đột biến với một xác suất;
- + Xác định quần thể mới x_i và hàm fitness f_i ;
- + Thay thế nếu các cá thể mới có fitness tốt hơn;
- + Ngược lại, giữ nguyên các cá thể và tiếp tục với các lần lặp tiếp theo.

Việc đầu tiên là lựa chọn toán tử, tiếp theo là đánh giá hàm thích nghi (fitness) và tái tạo quần thể tạo thành quá trình cơ bản của GA. Quá trình lặp lại cho đến khi đạt điều kiện dừng.



3.2 Các toán tử di truyền

Dạng thuật toán di truyền đơn giản nhất bao gồm ba loại toán tử: chọn lọc, lai ghép (tái tổ hợp) và đột biến [3].

3.2.1 Toán tử lai ghép

Lai ghép (tái tổ hợp) là một đặc điểm chính của thuật toán di truyền (GA). Phương pháp lai ghép 1 điểm cắt (tái tổ hợp đơn điểm) là dạng cơ bản nhất, trong đó một vị trí tái tổ hợp được chọn ngẫu nhiên và các phần phía sau vị trí đó của hai cá thể cha mẹ được hoán đổi để tạo ra hai cá thể con. Phương pháp này giúp tái kết hợp các khối xây dựng (schemas) trên các chuỗi khác nhau.

Toán tử này biểu diễn sự giao phối giữa các cá thể. Hai cá thể được chọn bằng toán tử chọn lọc và các vị trí lai ghép được chọn ngẫu nhiên. Sau đó, các gen tại các vị trí lai ghép này được trao đổi, do đó tạo ra một cá thể hoàn toàn mới (con cái). Toán tử lai ghép gần giống với sự tái tổ hợp sinh học giữa hai sinh vật có nhiễm sắc thể đơn (đơn bội).

Phép lai chéo: Toán tử này chọn ngẫu nhiên một locus và trao đổi các chuỗi con trước và sau locus đó giữa hai nhiễm sắc thể để tạo ra hai con.

Ví dụ các chuỗi 10000100 và 11111111 có thể được lai chéo sau locus thứ ba trong mỗi chuỗi để tạo ra hai chuỗi con 10011111 và 11100100.

Một phương pháp khác là tái tổ hợp "tham số hóa đồng nhất" (parameterized uniform crossover), trong đó mỗi vị trí bit có xác suất p (thường từ 0.5 đến 0.8) để được hoán đổi. Phương pháp này không có ưu tiên vị trí, nhưng có thể làm gián đoạn các tổ hợp allele đã thích nghi.

Việc chọn loại tái tổ hợp phù hợp phụ thuộc vào hàm fitness. Hiện tại, hai phương pháp phổ biến nhất là tái tổ hợp hai điểm và tái tổ hợp tham số hóa đồng nhất.

3.2.2 Toán tử đột biến

Trong các bài toán GA, đột biến thường được xem là vai trò thứ yếu, đóng vai trò đảm bảo rằng quần thể không cố định ở bất kỳ locus nào. Tuy nhiên, đột biến cũng rất quan trọng trong việc giải quyết các bài toán phức tạp, đặc biệt là khi GA phải tìm kiếm không gian rộng lớn.

Ý tưởng chính là chèn các gen ngẫu nhiên vào thế hệ con để duy trì sự đa dạng trong quần thể nhằm tránh sự hội tụ sớm. Toán tử này thường đảo ngẫu nhiên thứ tự một số gen trong nhiễm sắc thể.

Ví dụ với GA mã hoá nhị phân: chuỗi 00000100 có thể bị đột biến ở vị trí thứ hai để tạo ra 01000100.

Đột biến có thể xảy ra ở mỗi vị trí bit trong chuỗi với một xác suất nào đó, thường là rất nhỏ (ví dụ: 0,001).

3.2.3 Toán tử chọn lọc

Ý tưởng của toán tử này là ưu tiên những cá thể có điểm thể lực tốt và cho phép chúng truyền gen của mình cho các thế hệ tiếp theo. Toán tử này chọn lọc các nhiễm sắc thể trong quần thể để sinh sản. Nhiễm sắc thể càng phù hợp thì càng có nhiều khả năng được chọn để sinh sản. Một số loại toán tử chọn lọc:

- Chọn lọc xếp hạng (Rank selection): Sử dụng thứ hạng thay vì giá trị fitness để chọn các cá thể, giảm nguy cơ hội tụ sớm vào cực tiểu cục bộ.
- Chọn lọc giải đấu (Tournament selection): Các cá thể được chọn theo cặp và cá thể có fitness cao hơn được đưa vào nhóm của thế hệ tiếp theo.
- Chọn lọc ưu tú (Elitism selection): Đảm bảo rằng cá thể ưu tú trong một thế hệ luôn được truyền sang thế hệ tiếp theo. Nếu cá thể có giá trị fitness cao nhất không có mặt trong thế hệ tiếp theo sau quy trình chọn lọc thông thường, thì cá thể ưu tú cũng sẽ tự động được đưa vào thế hệ tiếp theo.
- Lựa chọn Boltzmann (Boltzmann selection): dựa trên phương pháp lấy mẫu và entropy, được sử dụng trong Mô phỏng Monte Carlo. Xác suất chọn được chuỗi tốt nhất là rất cao trong khi nó thực thi trong thời gian rất ngắn. Đây là phương pháp tốt để giảm nguy cơ mắc kẹt tại cực trị địa phương.

Tốc độ hội tụ của thuật toán di truyền phụ thuộc vào áp lực chọn lọc. Nếu áp lực chọn lọc quá cao, quần thể có thể mất tính đa dạng, dẫn đến hội tụ sớm, tức là thuật toán có thể bị mắc kẹt ở một giải pháp không tối ưu. Do đó, việc kiểm soát áp lực chọn lọc là rất quan trọng để đảm bảo rằng GA có thể tìm được giải pháp tối ưu toàn cục.

3.2.4 Một số toán tử khác

Ngoài tái tổ hợp và đột biến, còn nhiều toán tử khác đã được đề xuất để duy trì sự đa dạng trong quần thể, như:

- Crowding: Cá thể con thay thế cá thể trong quần thể gần giống nhất với nó.
- Fitness Sharing: Fitness của mỗi cá thể giảm dần dựa trên sự hiện diện của các cá thể khác có độ tương đồng cao.
- Hạn chế ghép đôi: Chỉ những cá thể đủ khác biệt hoặc đủ giống nhau mới được phép ghép đôi, để thúc đẩy sự hình thành các "loài" hoặc duy trì sự đa dạng.
- Mating Tags: Các phần của nhiễm sắc thể được sử dụng để nhận diện bạn đời tiềm năng. Chỉ những cá thể có tag tương ứng mới được phép ghép đôi.
- Một chiến lược khác là hạn chế ghép đôi trong không gian, nơi các cá thể chỉ có thể ghép đôi với hàng xóm gần nhất. Điều này giúp duy trì các loài biệt lập và thúc đẩy đổi mới tại các ranh giới giữa các loài.

3.3 Một số biến thể của thuật toán di truyền

Các biến thể của GA được phân loại rộng rãi thành năm loại chính, cụ thể là: mã hóa thực và nhị phân, đa mục tiêu, song song, hỗn loạn và lai [2].

3.3.1 Thuật toán di truyền mã hóa thực và nhị phân

Dựa trên cách biểu diễn nhiễm sắc thể, thuật toán di truyền (GA) được chia thành hai loại, cụ thể là GA mã hóa nhị phân và GA mã hóa thực.

a) Thuật toán di truyền mã hóa nhị phân

Biểu diễn nhị phân được sử dụng để mã hóa GA và được gọi là GA nhị phân, phương pháp này có độ phức tạp tính toán cao. Các toán tử di truyền cũng đã được điều chỉnh để thực hiện quá trình tìm kiếm. Mã hóa nhị phân là một trong những phương pháp mã hóa phổ biến trong GAs, đặc biệt là trong các bài toán có không gian tìm kiếm rời rạc.

Mã hóa nhị phân (tức là chuỗi bit) là mã hóa phổ biến nhất. Những lý thuyết GA hiện có được dựa trên giả định về mã hóa nhị phân có độ dài cố định và thứ tự cố định. Những lý thuyết đó có thể được mở rộng để áp dụng cho mã hóa không nhị phân, nhưng

những mở rộng này chưa phát triển tốt như lý thuyết ban đầu. Ngoài ra, các kinh nghiệm về cài đặt tham số phù hợp (ví dụ: cho tỷ lệ giao phối và tỷ lệ đột biến) nói chung đã được phát triển trong bối cảnh mã hóa nhị phân.

Một số ưu và nhược điểm:

- Tốc độ hội tụ nhanh: GAs mã nhị phân thường có tốc độ hội tụ nhanh và dễ dàng đạt được hội tụ sớm, giúp tìm ra giải pháp tối ưu trong thời gian ngắn [4].
- Tìm kiếm toàn cục: Chúng có khả năng tìm kiếm toàn cục, đơn giản trong thao tác và không yêu cầu đạo hàm, phù hợp với nhiều loại bài toán tối ưu hóa [5].
- Tuy nhiên, mã hóa nhị phân không tự nhiên, khó sử dụng cho nhiều vấn đề và chúng dễ bị sắp xếp khá tùy tiện.

b) Thuật toán di truyền mã hóa thực (Real-coded GA)

Các thuật toán di truyền mã hóa thực (RGAs) đã được sử dụng rộng rãi và ứng dụng nhiều trong thực tế. Biểu diễn nhiễm sắc thể liên quan chặt chẽ với các vấn đề trong đời sống thực. Chúng được áp dụng rộng rãi trong các bài toán tối ưu hóa có không gian tìm kiếm liên tục và thường được coi là phù hợp hơn cho các ứng dụng kỹ thuật thực tế.

Các chức năng chính của RGAs là mạnh mẽ, hiệu quả và chính xác. Tuy nhiên, RGAs gặp phải hiện tượng hội tụ sớm. Các nhà nghiên cứu đang làm việc để cải thiện hiệu suất của RGAs. Hầu hết RGAs được phát triển bằng cách thay đổi các toán tử giao phối, đột biến và chọn lọc.

Các toán tử di truyền bao gồm lai ghép và đột biến, là những yếu tố quyết định giúp GAs mã hóa thực có thể giải quyết các vấn đề tối ưu hóa phức tạp trong thực tế, dù chúng vẫn gặp phải một số hạn chế như hội tụ sớm và dính vào cực trị cục bộ.

3.3.2 Thuật toán di truyền đa mục tiêu

Thuật toán di truyền đa mục tiêu (MOGA) là phiên bản sửa đổi của thuật toán di truyền đơn giản (GA). MOGA khác GA ở chỗ phân bổ hàm thích nghi. Các bước còn lại giống với GA. Mục tiêu chính của MOGA là tạo ra Mặt Pareto tối ưu trong không gian mục tiêu sao cho không có bất kỳ sự cải thiện nào trong bất kỳ hàm thích nghi nào mà không làm ảnh hưởng đến các hàm thích nghi khác [6]. Hội tụ, đa dạng và bao phủ

là mục tiêu chính của GA đa mục tiêu. Các thuật toán di truyền đa mục tiêu được chia thành hai loại chính: dựa trên Pareto và dựa trên phân rã [7].

a) MOGA dựa trên Pareto

Khái niệm ưu thế Pareto được giới thiệu trong các thuật toán di truyền đa mục tiêu.

Patel et al [8] đề xuất một GA đa mục tiêu dựa trên phân rã (D-MOGA). Họ tích hợp học đối kháng (opposition-based learning) trong D-MOGA để tạo ra các vector trọng số. D-MOGA có khả năng duy trì sự cân bằng giữa đa dạng các nghiệm và khám phá không gian tìm kiếm.

b) MOGA dựa trên phân rã

Các thuật toán MOGA dựa trên phân rã phân rã vấn đề cho trước thành nhiều vấn đề con. Các vấn đề con này được giải quyết đồng thời và trao đổi giải pháp giữa các vấn đề con lân cận [7].

3.3.3 Thuật toán di truyền song song

Mục đích của GA song song là cải thiện thời gian tính toán và chất lượng của các giải pháp thông qua việc phân phối các cá thể.

Các thuật toán di truyền song song được phân thành ba loại chính: các thuật toán di truyền song song chủ-nô (master-slave), các thuật toán di truyền song song hạt mịn (fine grained), và các thuật toán di truyền song song hạt thô đa quần thể (multi-population coarse grained) [9].

a) GA song song chủ-nô

Tính toán hàm fitness được phân phối trên nhiều bộ xử lý. Số lượng các bộ xử lý được sử dụng trong GA song song chủ-nô (MS-PGA) lớn hơn so với các phương pháp khác. Việc tính toán hàm thích nghi có thể được tăng cường bằng cách tăng số lượng bộ xử lý.

b) GA song song hạt mịn

Máy tính song song được sử dụng để giải quyết các bài toán thực tế. Các toán tử di truyền bị giới hạn trong khu vực lân cận của chúng, nhưng cho phép có sự tương tác giữa các cá thể.

Trong vài thập kỷ qua, các nhà nghiên cứu đã làm việc về chính sách di cư của GA song song hạt mịn (FG-PGA).

c) GA song song hạt thô

Việc trao đổi các cá thể giữa các quần thể nhỏ được thực hiện, và các tham số điều khiển cũng được chuyển giao trong quá trình di cư.

GA song song hạt thô ít được sử dụng do tính phức tạp cao

3.3.4 Thuật toán di truyền hỗn loạn

Nhược điểm chính của GA là hội tụ quá sớm (premature convergence). Các hệ thống hỗn loạn được tích hợp vào GA để giải quyết vấn đề này. Sự đa dạng của GA hỗn loạn giúp loại bỏ hội tụ sớm. Các toán tử lai ghép và đột biến có thể được thay thế bằng các ánh xạ hỗn loạn (chaotic maps).

3.3.5 Thuật toán di truyền lai ghép

Các thuật toán di truyền có thể dễ dàng lai với các phương pháp tối ưu hóa khác để cải thiện hiệu suất của chúng như các phương pháp khử nhiễu hình ảnh, tối ưu hóa phản ứng hóa học, và nhiều hơn nữa. Các lợi ích chính của việc lai GA với các phương pháp khác là chất lượng giải pháp tốt hơn, hiệu quả hơn, đảm bảo các giải pháp khả thi và tối ưu hóa các tham số điều khiển [10]. Theo các nghiên cứu cho thấy khả năng lấy mẫu của GA bị ảnh hưởng nhiều bởi kích thước quần thể. Để giải quyết vấn đề này, các thuật toán tìm kiếm cục bộ như thuật toán memetic, Baldwinian, Lamarckian, và tìm kiếm cục bộ khác đã được tích hợp với GA. Một vấn đề khác trong GA là cài đặt tham số. Tìm các tham số điều khiển phù hợp là một nhiệm vụ khó khăn. Các kỹ thuật metaheuristic khác có thể được sử dụng với GA để giải quyết vấn đề này.

a) Tăng cường khả năng tìm kiếm

Các GA đã được tích hợp với các thuật toán tìm kiếm cục bộ để giảm sự "trôi dạt di truyền" (genetic drift). Toán tử tinh chỉnh rõ ràng đã được giới thiệu trong tìm kiếm cục bộ để tạo ra các giải pháp tốt hơn.

b) Tạo ra các giải pháp khả thi

Trong việc giải quyết các bài toán phức tạp và nhiều chiều, các toán tử di truyền của GA đôi khi tạo ra các giải pháp không khả thi trong một số miền bài toán cụ thể. Ví dụ như PMX crossover tạo ra các giải pháp không khả thi cho các vấn đề dựa trên thứ tự như bài toán TSP. Toán tử crossover bảo toàn khoảng cách đã được phát triển để tạo ra các giải pháp khả thi cho bài toán người bán hàng du lịch [11].

c) Thay thế các toán tử di truyền

Có khả năng thay thế các toán tử di truyền bằng các kỹ thuật tìm kiếm khác. Toán tử đột biến tiêu chuẩn đã được thay thế bằng quá trình làm nguội mô phỏng trong [12].

Các khái niệm cơ bản của tính toán lượng tử được sử dụng để nâng cao hiệu suất của GA. Các toán tử heuristic crossover và leo đồi (hill-climbing) có thể được tích hợp vào GA để giải quyết three-matching problem.

d) Tối ưu hóa các tham số điều khiển

Các tham số điều khiển của GA đóng vai trò quan trọng trong việc duy trì sự cân bằng giữa sự tăng cường và sự đa dạng hóa. GA có thể được sử dụng để tối ưu hóa các tham số điều khiển của các kỹ thuật khác.

Các thuật toán di truyền đã được sử dụng để tối ưu hóa tốc độ học, trọng số và cấu trúc của mạng nơ-ron [13].

4 Thực nghiệm

4.1 Bộ dữ liệu

Tiến hành thực nghiệm trên bộ dữ liệu gồm 4 file CSV chứa toạ độ của các thành phố (x, y) do Maxwell thu thập và phát triển:

- Dữ liệu siêu nhỏ: cỡ 10 thành phố
- Dữ liệu nhỏ: cỡ 30 thành phố
- Dữ liệu vừa: cỡ 100 thành phố
- Dữ liệu lớn: cỡ 1000 thành phố

Trong lần thực nghiệm này, với bài toán sắp xếp thứ tự các thành phố trên đường đi, GA mã hoá thực (RGAs) được lựa chọn vì tính đơn giản và trực quan của phương pháp này.

4.2 Các bước xây dựng giải thuật GA

Một nhiễm sắc thể (hay một lời giải chấp nhận được) là một danh sách có thứ tự các thành phố sẽ đi qua. Ví dụ: [0,3,7,1,5,8,6,2].

Bước 1: Đọc dữ liệu đầu vào và tạo ma trận khoảng cách giữa các thành phố. Khoảng cách giữa 2 thành phố $A(x_A, y_A)$, $B(x_B, y_B)$ bất kỳ ở đây được tính theo công thức chuẩn Euclid:

$$d(A, B) = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

Bước 2: Định nghĩa hàm thích nghi $fitness()$ nhận đầu vào là một NST. Hàm thích nghi có giá trị càng cao thì NST đó càng "tốt".

Với NST $X = \{X_i, i = \overline{1, n}\}$, đánh giá độ thích nghi thông qua hàm thích nghi được lựa chọn như sau:

$$fitness_{NST} = \frac{1}{\sum_{i=0}^{n-1} d(X_i, X_{i+1})}$$

Bước 3: Khởi tạo quần thể.

Algorithm 2 Initialization population (Khởi tạo quần thể)

```
0: Function InitializePopulation(cities)
0: population  $\leftarrow []$ 
0: num_cities  $\leftarrow \text{length}(\text{cities})$ 
0: pop_size  $\leftarrow 100$ 
0: for each individual in range(pop_size) do
0:   individual  $\leftarrow \text{list}(\text{range}(\text{num\_cities}))$ 
0:   shuffle(individual)
0:   distance  $\leftarrow \text{fitness}(\text{individual}, \text{distance\_mat}=\text{distance\_matrix}(\text{cities}))$ 
0:   append(population, [distance, individual])
0: end for
0: return population = 0
```

Số lượng NST càng nhiều thời gian xử lý, tính toán càng kéo dài, số lượng NST nếu quá ít dễ thiếu sự đa dạng trong quần thể, dẫn tới tình trạng mắc kẹt trong cực trị địa phương. Vì vậy, trong quá trình thực nghiệm, sau một vài phép thử với số lượng NST khác nhau, chúng tôi cố định số lượng NST trong quần thể là 100.

Bước 4: Định nghĩa phép các toán tử chọn lọc, lai ghép và đột biến.

- **Chọn lọc:** Sử dụng phương pháp chọn lọc giải đấu (Tournament Selection). Trong giải thuật di truyền, sử dụng chọn lọc giải đấu tổng thể : bằng cách chia quần thể thành từng nhóm nhỏ và so sánh độ thích nghi của các NST trong từng nhóm, từ đó chọn ra NST "tốt nhất". NST chiến thắng trong nhóm sẽ được chọn để tiếp tục vào giai đoạn lai ghép và tạo ra thế hệ mới. Lặp lại quá trình này cho đến khi chọn

đủ số cá thể cần thiết cho bước lai ghép. Đây là một phương pháp hiệu quả để chọn lọc các NST trong thuật toán di truyền, với khả năng kiểm soát áp lực chọn lọc thông qua việc điều chỉnh kích thước giải đấu. Áp lực chọn lọc cao, các cá thể NST có fitness cao có nhiều khả năng được chọn hơn, có thể làm giảm sự đa dạng của quần thể.

Algorithm 3 Tournament Selection

Require: population, tournament_size, population_size

Ensure: selected_individuals

```
1: selected_individuals  $\leftarrow$  []
2: for each individual in population do
3:   tournament_contestants  $\leftarrow$  random selection of tournament_size individuals from
     population
4:   winner  $\leftarrow$  best individual by fitness in tournament_contestants
5:   selected_individuals.append(winner)
6: end for
7: return selected_individuals
```

- **Lai ghép:** Thực hiện và so sánh giữa 3 phương pháp lai ghép: Lai ghép 1 điểm cắt, lai ghép k điểm cắt (2 điểm cắt) và lai ghép PMX.
- **Đột biến:** Thực hiện hoán đổi vị trí của 2 gen (thành phố) bất kì trong NST cha để tạo NST mới. Điều này nhằm tạo ra sự thay đổi ngẫu nhiên trong quần thể, giúp khám phá không gian tìm kiếm rộng hơn từ đó giảm khả năng mắc kẹt tại cực trị địa phương.

Bước 5: Khởi tạo hàm GA.

Đầu vào của hàm gồm các thông số: danh sách các thành phố cần tối ưu hóa (cities), quần thể ban đầu (population), số lượng thành phố, kích thước quần thể, số thế hệ (điều kiện dừng thuật toán) - cố định là 30, tỷ lệ lai ghép, tỷ lệ đột biến, hàm lai ghép (bao gồm lai ghép 1 điểm cắt, lai ghép nhiều điểm cắt và lai ghép PMX), và số lượng cá thể được chọn ngẫu nhiên để tìm cha mẹ tốt nhất (k).

Thuật toán khởi đầu bằng việc xác định cá thể tốt nhất (elite) từ quần thể và tính toán ma trận khoảng cách giữa các thành phố. Trong mỗi thế hệ, một danh sách cá thể mới (new_individuals) được tạo ra. Đối với mỗi cặp cá thể, nếu giá trị ngẫu nhiên nhỏ hơn crossover_rate, hai cá thể cha mẹ tốt nhất sẽ được chọn từ k cá thể ngẫu nhiên, sau đó lai ghép bằng hàm crossover_func để tạo hai cá thể con. Nếu giá trị ngẫu nhiên nhỏ hơn

mutation_rate, các cá thể con sẽ được đột biến. Nếu không xảy ra lai ghép, hai cá thể mới sẽ được tạo ngẫu nhiên. Tất cả cá thể con được thêm vào danh sách new_individuals. Sau đó, quần thể được cập nhật thông qua chọn lọc giải đấu, và cá thể tốt nhất (elite) được cập nhật nếu cần.

Algorithm 4 Genetic Algorithm (GA)

Require: cities, population, lenCities, population_size, num_of_generations, crossover_rate, mutation_rate, crossover_func, k

Ensure: sorted population, elite

```
1: elite  $\leftarrow$  best individual in population
2: distance_mat  $\leftarrow$  distance_matrix(cities)
3: for each generation do
4:   new_individuals  $\leftarrow$  []
5:   for 1 to population_size / 2 do
6:     if random  $\leq$  crossover_rate then
7:       father1  $\leftarrow$  best of  $k$  random individuals
8:       father2  $\leftarrow$  best of  $k$  random individuals
9:       inv1, inv2  $\leftarrow$  crossover_func(father1, father2)
10:    if random  $\leq$  mutation_rate then
11:      inv1  $\leftarrow$  mutation(inv1)
12:      inv2  $\leftarrow$  mutation(inv2)
13:    end if
14:  else
15:    inv1, inv2  $\leftarrow$  random individuals
16:  end if
17:  add inv1 and inv2 to new_individuals
18: end for
19: population  $\leftarrow$  tournaments(population, new_individuals)
20: update elite with best individual
21: end for
22: return sorted population, elite = 0
```

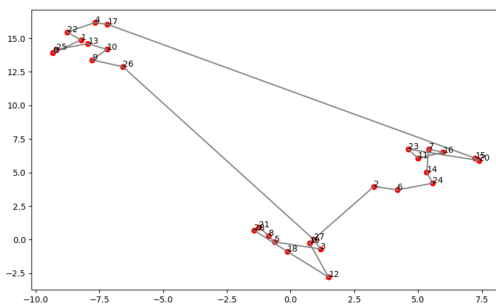
5 Đánh giá và kết luận

Sau khi thực nghiệm thuật toán với 4 datasets cỡ từ nhỏ đến lớn, thu được kết quả và đánh giá qua bảng sau:

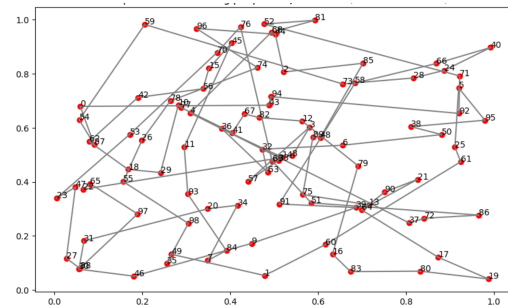
Data	Tiny	Small	Medium	Large
Số thành phố	10	30	100	1000
Số lần lặp	30	30	30	30
Thời gian sử dụng lại ghép 1 điểm cắt	0.84	1.71	6.8	341.24
Thời gian sử dụng lại ghép 2 điểm cắt	0.66	1.34	5.64	187.6
Thời gian sử dụng lại ghép PMX	0.65	1.21	3.95	
Độ thích nghi lớn nhất	0.08726	0.01913	0.05302	0.0023
Độ dài chu trình ngắn nhất	11.46	52.28	18.86	358.31

Bảng 1: Caption

Có thể thấy, khi tăng tích cỡ bộ dữ liệu (số lượng các thành phố) lên nhiều lần, thời gian tính toán và độ phức tạp của mỗi vòng lặp tăng lên đáng kể. Vì vậy, với những dataset cỡ chục nghìn (10000- 20000 thành phố) vẫn đang là bài toán khó và cần kết hợp với các phương pháp khác để giảm thời gian và khối lượng tính toán. Sau 30 thế hệ (vòng lặp), ta thu được các đường đi có độ dài ngắn nhất (NST có hàm thích nghi cao nhất) như sau:



(a) Data small



(b) Data medium

5.1 So sánh hiệu quả giữa các phép lai ghép

5.2 Một số phương án kết hợp đề xuất

5.2.1 Thuật toán tìm kiếm Tabu

Giải thuật tìm kiếm địa phương đưa ra một lời giải hợp lệ ban đầu cho bài toán và xét các lời giải láng giềng với hy vọng tìm kiếm một lời giải tốt hơn lời giải hiện tại. Tìm kiếm địa phương có xu hướng tối ưu cục bộ hoặc lặp rất nhiều lần khi các lời giải láng giềng là như nhau. Tabu search nâng cao hiệu suất tìm kiếm địa phương bằng cách nói lỏng quy tắc cơ bản của nó. Đầu tiên, tabu search cho phép di chuyển đến một lời giải tồi

hơn nếu như không thể tìm được một lời giải láng giềng tốt hơn lời giải hiện tại (trường hợp đạt đến tối ưu cục bộ). Thứ hai, nếu một lời giải tiềm năng đã được truy cập trước đó trong một khoảng thời gian ngắn nhất định thì lời giải này được đưa vào danh sách cấm, không được phép di chuyển từ lời giải hiện tại đến một lời giải nằm trong danh sách cấm.

Một số phương án kết hợp GA và Tabu:

- Áp dụng Tabu Search cho mỗi cá thể trong quần thể của GA. Khi một cá thể được chọn để lai ghép hoặc đột biến, chạy Tabu Search trên cá thể đó để cải thiện giải pháp trước khi đưa nó vào thế hệ mới.
- Sử dụng Tabu Search để tối ưu hóa quần thể GA trong suốt quá trình tìm kiếm. Điều này có thể bao gồm việc áp dụng Tabu Search cho các giai đoạn tìm kiếm quần thể, giúp điều chỉnh động học quần thể hoặc kiểm soát việc chọn lọc. Sau một số thế hệ nhất định, sử dụng Tabu Search để điều chỉnh lại các cá thể trong quần thể, có thể loại bỏ những giải pháp đã tìm kiếm gần đây hoặc tránh bị mắc kẹt trong cực trị cục bộ.

5.2.2 Tìm kiếm địa phương kết hợp

Tài liệu

- [1] I. Smith. *Overview Evolutionary Algorithms*. ScienceDirect. 2011. DOI: [10.1016/j.knosys.2011.04.003](https://doi.org/10.1016/j.knosys.2011.04.003). URL: https://www.sciencedirect.com/science/article/pii/S0957417411004490?ref=pdf_download&fr=RR-2&rr=8f3dd8ba3894821d.
- [2] Qi Zhang A Yang Xiang. *Deep Learning and Parallel Computing Environment for Bioengineering Systems*. Accessed: 2024-12-18. Elsevier, 2020. **chapter** 13.3 Evolutionary Algorithms. DOI: [10.1016/B978-0-12-816718-2.00013-3](https://doi.org/10.1016/B978-0-12-816718-2.00013-3). URL: <https://www.sciencedirect.com/book/9780128167182/deep-learning-and-parallel-computing-environment-for-bioengineering-systems>.
- [3] Melanie Mitchell. *An Introduction to Genetic Algorithms*. Fifth printing, 1999 (First MIT Press paperback edition, 1998). Includes bibliographical references and index. Cambridge, Massachusetts, London, England: A Bradford Book, The MIT Press, 1996, **pages** 8, 8, 117–118, 128–130. ISBN: 0-262-13316-4 (HB), 0-262-63185-7 (PB). URL: <https://mitpress.mit.edu/9780262631853/an-introduction-to-genetic-algorithms/>.
- [4] Luo Pi **and others**. “An improved genetic algorithm and its performance analysis”. *in Proceedings of the 2001 International Conferences on Info-Tech and Info-Net (ICII 2001)*: **volume** 4. 2001, **pages** 316–321. DOI: [10.1109/ICII.2001.983840](https://doi.org/10.1109/ICII.2001.983840). URL: <https://ieeexplore.ieee.org/document/983840>.
- [5] Jun Zhang, Henry Shu-Hung Chung **and** Guangyu Zhong. “Comparison of Binary Code and Real Code in the Genetic Algorithm”. *in Proceedings of the 2004 International Conference on Machine Learning and Cybernetics*: **volume** 1. 2004, **pages** 242–247. DOI: [10.1109/ICMLC.2004.1382287](https://doi.org/10.1109/ICMLC.2004.1382287). URL: <https://ieeexplore.ieee.org/document/1382287>.
- [6] Bin Li **and others**. “Many-objective evolutionary algorithms: A survey”. *in ACM Computing Surveys*: 47.2 (2015), **pages** 1–38. DOI: [10.1145/2792984](https://doi.org/10.1145/2792984).
- [7] Matias T. M. Merich **and** Art J. Deutz. “A tutorial on multiobjective optimization: fundamentals and evolutionary methods”. *in Natural Computing*: 17.3 (2018), **pages** 585–609. DOI: [10.1007/s11047-018-9723-5](https://doi.org/10.1007/s11047-018-9723-5).

- [8] R. Patel, M. M. Raghuwanshi **and** L. G. Malik. “Decomposition Based Multi-Objective Genetic Algorithm (DMOGA) with Opposition Based Learning”. **in** *Journal of Applied Soft Computing*: (2012).
- [9] T. Harada **and** E. Alba. “Parallel Genetic Algorithms: A Useful Survey”. **in** *ACM Computing Surveys*: 53.4 (2020), **pages** 1–39.
- [10] T. A. El-Mihoub **and others**. “Hybrid Genetic Algorithms: A Review”. **in** *Engineering Letters*: 13.2 (2006).
- [11] B. Freisleben **and** P. Merz. “New Genetic Local Search Operators for the Traveling Salesman Problem”. **in** *Proceedings of the Fourth Conference on Parallel Problem Solving from Nature*: **by editor** H.-M. Voigt **and others**. **volume** 1141. Lecture Notes in Computer Science. Berlin, Germany: Springer-Verlag, 1996, **pages** 890–899.
- [12] K. C. Tan **and others**. “System Identification and Linearisation Using Genetic Algorithms with Simulated Annealing”. **in** *Proceedings of the First IEE/IEEE International Conference on Genetic Algorithms in Engineering Systems: Innovations and Applications (GA in Eng. Syst.)* Sheffield, UK, 1995, **pages** 164–169.
- [13] N. Chaiyaratana **and** A. M. Zalzala. “Hybridisation of Neural Networks and a Genetic Algorithm for Friction Compensation”. **in** *Proceedings of the 2000 Congress on Evolutionary Computation (CEC 2000)*: **volume** 1. San Diego, USA, 2000, **pages** 22–29.
- [14] Saloni Gupta **and** Dr. Panwar. “Solving Travelling Salesman Problem Using Genetic Algorithm”. **in** *International Journal of Advanced Research in Computer Science and Software Engineering*: 3 (2013), **pages** 376–380. URL: https://www.researchgate.net/publication/280978365_Solving_Travelling_Salesman_Problem_Using_Genetic_Algorithm.
- [15] Q. Nguyễn Trung **and** Đức Nguyễn Trọng. “Song song hóa thuật toán lai ghép Davis’ Order Crossover trên FPGA sử dụng True Dual Port RAM - Một cách tiếp cận trong giải quyết bài toán người du lịch bằng giải thuật di truyền”. **in** *JMST*: 69.PH 69 (2022), **pages** 72–76. URL: <https://www.jmst.vimaru.edu.vn/index.php/tckhcnhh/article/view/35>.

- [16] Philip Adewole, Adio Akinwale **and** Kehinde Otubamowo. “A Genetic Algorithm for Solving Travelling Salesman Problem”. **in** *International Journal of Advanced Computer Sciences and Applications*: 2 (2011). DOI: [10 . 14569 / IJACSA . 2011 . 020104](https://doi.org/10.14569/IJACSA.2011.020104). URL: [https : / / www . researchgate . net / publication / 49613263 _ A _ Genetic_Algorithm_for_Solving_Travelling_Salesman_Problem](https://www.researchgate.net/publication/49613263_A_Genetic_Algorithm_for_Solving_Travelling_Salesman_Problem).